

IntelliCode

A Context-Aware Development Environment with Persistent Project Intelligence

Project Synopsis

1. Introduction and Problem Statement

Modern development environments can work with multiple coding models and external tools, but developers routinely face three practical problems that reduce productivity and increase the cost of iteration.

Context Loss on Model Switching: When a developer switches from one AI coding model to another, the new model typically has to re-read the entire project from scratch. This wastes tokens and time, and project context is never efficiently shared between models.

No Built-in Change Impact Analysis: When a file or function is modified (for example, `authService.ts`), it is difficult to immediately know which other files or components — such as the login page, user service, or session manager — may be affected. Developers must manually trace these dependencies.

Complicated MCP / API Integration: Connecting external tools and services requires finding the correct MCP server, configuring it, adding credentials, managing API keys, and repeating this process for every project. Tool and service integration is therefore neither simple nor centralized.

2. Proposed Solution

IntelliCode creates a persistent, structural understanding of a project at the start of development and continuously updates it as the project evolves. When a developer initializes a project with a Master Prompt — describing requirements, architecture, and goals — the system analyzes the source code, builds an Abstract Syntax Tree (AST), extracts symbols, and constructs a dependency graph. Together these form a Project Knowledge Graph, which becomes a shared project-intelligence layer accessible to any connected model or tool.

2.1 Persistent Context Across Models

Instead of every model re-reading the whole project, GPT, Gemini, and Claude (or other connected models) all draw on the same Persistent Project Context. When a developer switches models mid-task, the new model continues from the existing project state rather than starting over, resulting in fewer repeated file reads, fewer unnecessary tokens, and faster continuation of work.

2.2 Change Impact Analysis (CIA)

The Knowledge Graph maintains relationships between files, functions, classes, modules, and their dependencies. When a component changes, the system traces the graph to identify potentially affected components and relevant tests — for example, a change to `paymentService.ts` might flag `Cart`, `Checkout`, and `Order` as affected, and recommend running payment and checkout tests — before the change causes unexpected problems downstream.

2.3 Centralized MCP and Integration Layer

An Integration Hub manages MCP servers (such as GitHub, database, and browser MCPs), API keys, external services, tool permissions, and project-specific configurations from a single place, replacing the current fragmented, repeat-every-project setup process.

3. System Flow and Core Modules

At a high level, the system flow proceeds as follows: a Master Prompt triggers Project Analysis, which produces an AST, symbol table, and dependency data; these are consolidated into the Project Knowledge Graph and State. Context Retrieval then supplies relevant context to the active Task or Chat, which is handled by the Model/Tool Layer. Resulting code changes pass through Change Impact Analysis and Test/Validation, after which the Project State is updated before the next task begins. A parallel Integration Layer manages MCP connections, API keys, and external services throughout.

Module	Main Function
Project Analyzer	Builds the AST, symbol table, and dependency data for the project.
Project Knowledge Graph	Maintains relationships between files, functions, classes, and modules.
Persistent Context Engine	Maintains reusable project- and task-level context across sessions.
Model Gateway	Allows different development models to work from the same project state.
Change Impact Analysis	Identifies components potentially affected by a given change.
Integration Hub	Manages MCP servers, API keys, and external services centrally.
Validation Engine	Runs tests and validates changes against the affected components.

4. What Is New in This System

The contribution is not simply the use of a coding model, but a layer built around development models that provides:

- **Persistent Project Intelligence:** Project understanding is maintained independently of any single model or session.
- **Cross-Model Context Continuity:** Different models can work on the same project without rebuilding understanding from zero.
- **Automated Change Impact Analysis:** Project relationships are used to identify components affected by a change.
- **Unified Tool Integration:** MCP servers, APIs, and external services are managed through one centralized layer.

5. Evaluation Plan

- **Experiment 1 — Model Switching:** Compare full project re-reading versus persistent project context, measuring context size, tokens used, time taken, and task-continuation success.
- **Experiment 2 — Change Impact Analysis:** Make controlled changes and compare actual affected components against components identified by IntelliCode.
- **Experiment 3 — Integration Workflow:** Measure effort for manual MCP/API configuration versus the IntelliCode integration workflow.
- **Experiment 4 — Development Task:** Give the same development task under a conventional approach and under IntelliCode, comparing context usage and completion time.

6. Expected Benefits and Conclusion

IntelliCode aims to deliver: reduced repeated context processing, lower unnecessary token consumption, faster model switching, better understanding of project dependencies, early identification of potentially affected components, simpler MCP/API integration, and a persistent project state across development tasks.

In summary: IntelliCode is a development environment that builds a persistent structural understanding of a project once, keeps that understanding updated, and uses it to provide context continuity across models, change-

impact analysis, and centralized tool integration — built around three core ideas: Persistent Project Context, Change Impact Analysis, and Unified MCP/API Integration.